Inverse Kinematics Design

Inverse Kinematics Design

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

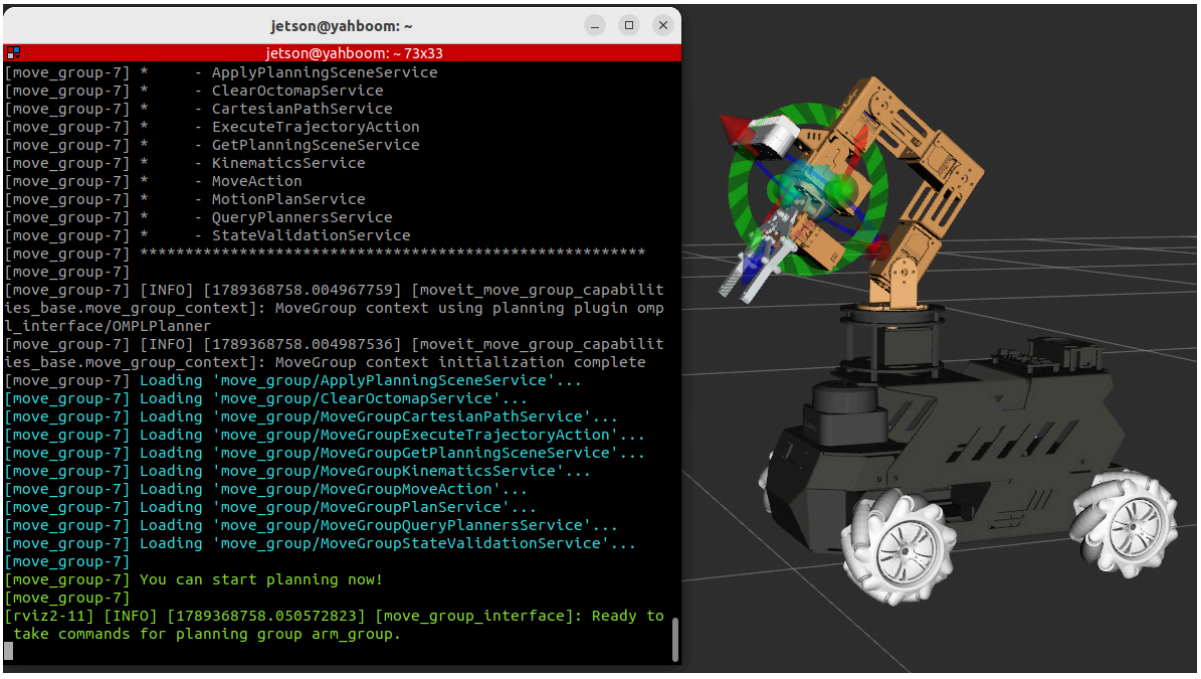
1. Course Content

This course explains how to call the functions in the MoveIt library to implement inverse kinematics design of the robot arm. Inverse kinematics design refers to computing the required joint angles based on the desired pose of the end effector. Here, in RViz, we specify a pose for the end effector, and then let MoveIt2 plan and move to this pose.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:

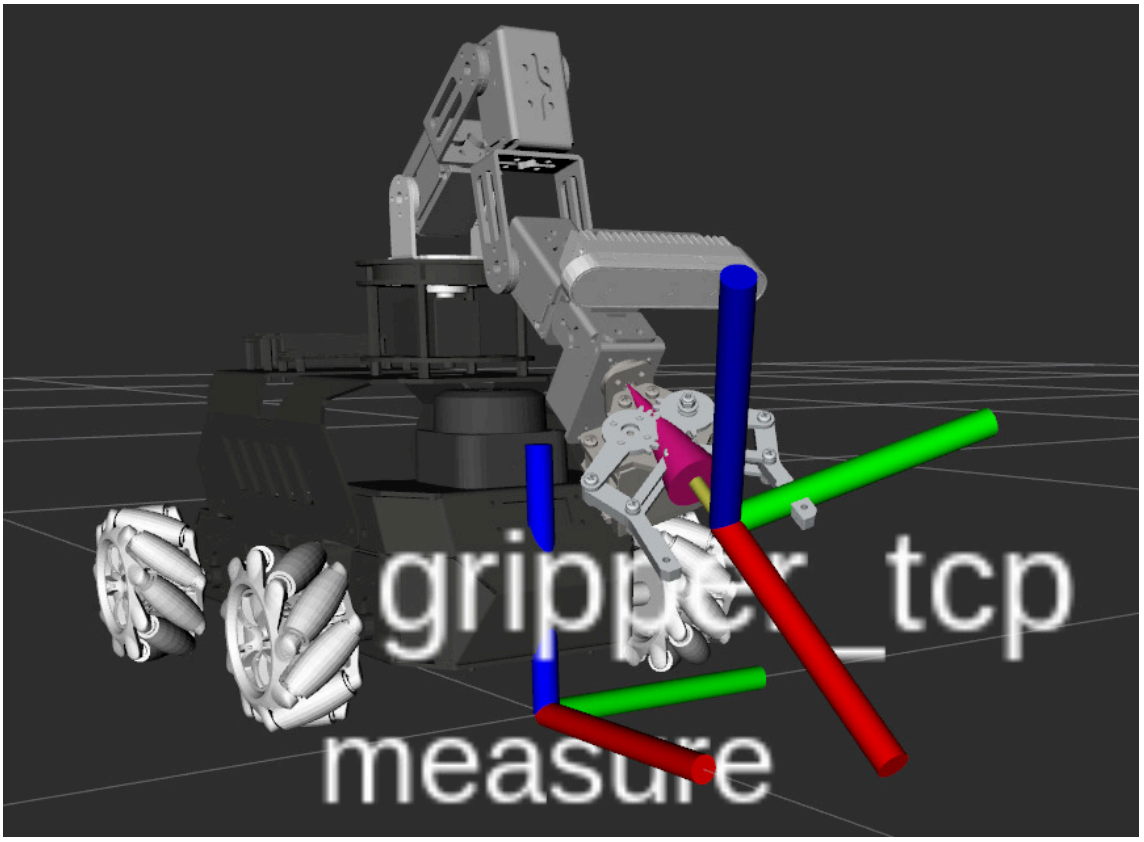


3. Case Demonstration

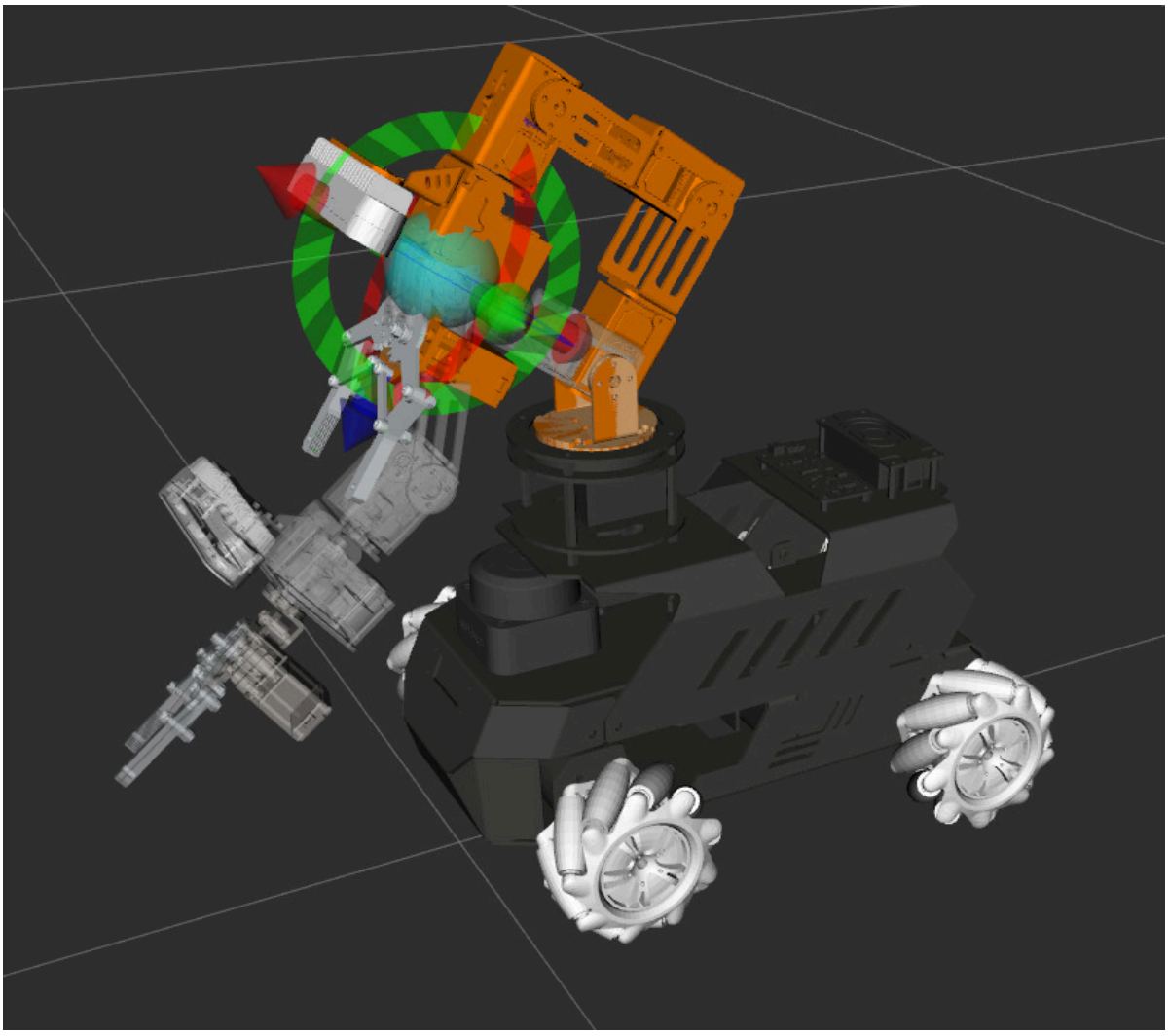
```
ros2 run moveit_demo set_target_position --ros-args -p x:=0.17 -p y:=0.0 -p
z:=0.1
```

> **Tip**

- x:=0.17, y:=0.0, z:=0.1 are the end-effector positions used as a reference for the demonstration case. You can specify the end-effector position yourself through parameters. If the end-effector position is kinematically unreachable, the program will give a prompt: the target point cannot be reached.
- The reference coordinate frame of the end effector is the measure coordinate frame, whose origin is the projection point on the ground of the center point of the car's front.



After the program runs, the robot arm in RViz will plan and move to the pose we set, as shown in the figure below:



```
[INFO] [1789372677.080320134] [move_group_interface]: Planning request accepted
[INFO] [1789372677.174153239] [move_group_interface]: Planning request complete!
[INFO] [1789372677.174659466] [move_group_interface]: time taken to generate plan: 0.0709334 seconds
[INFO] [1789372677.174770542] [set_target_position]: Planning succeeded, executing.
[INFO] [1789372677.176885950] [move_group_interface]: Execute request accepted
[INFO] [1789372680.516291942] [move_group_interface]: Execute request success!
[INFO] [1789372680.516618931] [set_target_position]: Execution complete, final joint angles:
[INFO] [1789372680.516742743] [set_target_position]:   arm1 = 1.5719 rad (90.06 deg)
[INFO] [1789372680.516802586] [set_target_position]:   arm2 = 0.6197 rad (35.51 deg)
[INFO] [1789372680.516839323] [set_target_position]:   arm3 = -0.3973 rad (-22.76 deg)
[INFO] [1789372680.516868764] [set_target_position]:   arm4 = 2.2608 rad (129.53 deg)
[INFO] [1789372680.516903294] [set_target_position]:   arm5 = 1.9477 rad (111.60 deg)
```

4. Core Source Code Analysis

Program source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/set_target_position.cpp
```

initialize function description:

- `declare_parameter("x", 0.12)` / `"y"` / `"z"`: Declares the three parameters x, y, z of the target position (unit: meters), with default values of 0.12, 0.0, and 0.15 respectively, which can be overridden via the `-p` command line.

- `get_parameter("x").as_double()`: Gets the parameter value and converts it to the double type.
- `MoveGroupInterface(shared_from_this(), "arm_group")`: Creates a MoveGroup control interface, bound to the `arm_group` planning group.
- `setNumPlanningAttempts(10)`: Sets the number of planning attempts per call to 10.
- `setPlanningTime(10.0)`: Sets the maximum time for a single planning attempt to 10 seconds.
- `setMaxVelocityScalingFactor(1.0)` / `setMaxAccelerationScalingFactor(1.0)`: Both velocity and acceleration scaling factors are 1.0, using the full speed of the planned motion.
- `setEndEffectorLink("gripper_tcp")`: Specifies the end effector link as `gripper_tcp`, i.e., the center point of the gripper.
- `setPoseReferenceFrame("measure")`: Sets the pose reference coordinate frame to `measure`; all target positions are relative to this frame.
- `setGoalJointTolerance(0.01)`: Sets the joint angle tolerance to 0.01 rad, i.e., the allowable range of joint angle error when reaching the target.
- `setGoalPositionTolerance(0.005)`: Sets the position tolerance to 0.005 m, i.e., the allowable range of end-effector position error.
- `setPositionTarget(tx, ty, tz)`: Sets the target position of the end effector (position only, no orientation specified); MoveIt2 will automatically solve the inverse kinematics to obtain the joint angles.
- `plan(plan)`: Plans a trajectory for the target pose, internally calling the inverse kinematics to solve for the joint angles.
- `execute(plan)`: If planning succeeds, executes the trajectory and drives the robot arm to the target pose.
- `plan.trajectory_.joint_trajectory.points`: Gets the sequence of joint trajectory points from the planning result.
- `points.back()`: Takes the last point of the trajectory, i.e., the final joint angles when reaching the target.
- `last.positions[i] * 180.0 / M_PI`: Converts the final joint angles from radians to degrees and prints them.
- `rclcpp::shutdown()`: Shuts down the node after the task is complete.

```
void initialize()
{
  // ===== 1. Parameters: target position (xyz only, no orientation) =====
  this->declare_parameter("x", 0.12);
  this->declare_parameter("y", 0.0);
  this->declare_parameter("z", 0.15);

  double tx = this->get_parameter("x").as_double();
  double ty = this->get_parameter("y").as_double();
  double tz = this->get_parameter("z").as_double();

  RCLCPP_INFO(this->get_logger(),
    "Target position (measure frame): [%.4f, %.4f, %.4f]", tx, ty, tz);
```

```
    // ===== 2. MoveGroupInterface =====
    move_group_interface_ =
      std::make_shared<moveit::planning_interface::MoveGroupInterface>(
        shared_from_this(), "arm_group");

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(10.0);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);
    move_group_interface_->setEndEffectorLink("gripper_tcp");
    move_group_interface_->setPoseReferenceFrame("measure");
    move_group_interface_->setGoalJointTolerance(0.01);
    move_group_interface_->setGoalPositionTolerance(0.005);

    move_group_interface_->setPositionTarget(tx, ty, tz);

    RCLCPP_INFO(this->get_logger(), "Starting planning...");
    moveit::planning_interface::MoveGroupInterface::Plan plan;
    if (move_group_interface_->plan(plan) ==
        moveit::core::MoveItErrorCode::SUCCESS)
    {
      RCLCPP_INFO(this->get_logger(), "Planning succeeded, executing.");
      move_group_interface_->execute(plan);

      // Print final joint angles
      const auto& points = plan.trajectory_.joint_trajectory.points;
      if (!points.empty())
      {
        const auto& names = plan.trajectory_.joint_trajectory.joint_names;
        const auto& last = points.back();
        RCLCPP_INFO(this->get_logger(), "Execution complete, final joint
angles:");
        for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
        {
          RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
            names[i].c_str(), last.positions[i],
            last.positions[i] * 180.0 / M_PI);
        }
      }
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(),
        "Planning failed! Target position may be out of workspace or cause
collision.");
    }

    rclcpp::shutdown();
  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
    move_group_interface_;
};
```